

基于故障信息的 SOSEMANUK 猜测确定攻击

陈浩¹ 王韬¹ 张帆² 赵新杰³

(1 解放军军械工程学院信息工程系, 河北 石家庄 050003; 2 浙江大学信息与电子工程学系, 浙江 杭州 310027; 3 北方电子设备研究所, 北京 100830)

摘要 针对 SOSEMANUK 流密码已有攻击方法复杂度过高的不足, 提出并讨论了一种基于故障信息的猜测确定攻击方法。首先利用代数方法构建密码在比特层面的等效代数方程组, 然后向密码注入随机单字故障, 在深入分析故障传播特征的基础上, 将故障信息表示成代数方程组并猜测密码部分内部状态, 使用 CryptoMinisat 解析器求解代数方程组恢复密码初始内部状态。实验结果表明: 对密码首轮加密进行攻击, 恢复密码全部初始内部状态所需的故障注入次数为 20 次, 计算复杂度为 $O(2^{96})$, 对密码前两轮加密进行攻击, 无须猜测密码内部状态, 仅注入 10 个单字故障即可恢复密码全部初始内部状态。与已有结果相比, 新方法攻击复杂度显著降低。

关键词 流密码; SOSEMANUK; 猜测确定攻击; 故障注入; CryptoMinisat 解析器

中图分类号 TN915; TP393 **文献标志码** A **文章编号** 1671-4512(2017)02-0072-06

Fault-based guess-and-determine attack on SOSEMANUK

Chen Hao¹ Wang Tao¹ Zhang Fan² Zhao Xinjie³

(1 Department of Information Engineering, Ordnance Engineering College, Shijiazhuang 050003, Hebei China; 2 Department of Information Science and Electrical Engineering, Zhejiang University, Hangzhou 310027, China; 3 Institute of North Electronic Equipment, Beijing 100830, China)

Abstract The SOSEMANUK stream cipher is a member of the finalists of the eSTREAM project. In this paper, the previous known attacks against SOSEMANUK was presented and discussed. Firstly, SOSEMANUK was described as a set of equations involving the public and key variables at bit level. Secondly, the attacker was assumed to be able to fault a random inner state word and the faults were described as a set of equations by analyzing the propagation of faults. Thirdly, the CryptoMinisat solver was adapted to recover the secret inner state by guessing certain inner state words and solving the combined equations. The results show that the first round attack recovers the secret internal states, requires 20 faults and the computational complexity is dramatically reduced to $O(2^{96})$. The first two rounds attack recovers the whole states, requires 10 faults without guessing any inner state word, which is better than the previous known cryptanalytic results.

Key words stream cipher; SOSEMANUK; guess-and-determine attack; fault injection; CryptoMinisat solver

SOSEMANUK 流密码^[1] 作为欧洲 eSTREAM 计划^[2] 软件组最终胜选的四流密码算法之一, 其安全性研究一直是密码界的研究热点。

文献[3]分别从字、字节和比特层面给出了针对 SOSEMANUK 流密码的猜测确定攻击, 所需的计算复杂度分别为 $O(2^{224})$, $O(2^{176})$ 和 $O(2^{192})$ 。然

收稿日期 2016-03-15.

作者简介 陈浩(1987-), 男, 博士研究生, E-mail: chenhao81823264@163.com.

基金项目 国家自然科学基金资助项目(61173191, 61272491, 61309021); 中央高校基本科研业务费专项资金资助项目(2015QNA5005).

而,由于上述攻击方法均未充分利用密码差分 S 盒的分布特性,因此导致攻击所需复杂度较高.文献[4-6]基于单字随机故障模型,对 SOSEMANUK 流密码提出一种差分故障攻击方法,恢复密码全部初始内部状态须要注入 6 144 个故障,计算复杂度为 $O(2^{48})$,但由于该攻击方法仅利用注入到密码内部状态 s_4, s_5 的随机故障,故障信息并不能扩展至密码所有初始内部状态,须猜测部分密码内部变量才能恢复密码全部初始内部状态,从而导致攻击计算复杂度较高.此外,该攻击方法依赖手工故障传播分析和密钥信息推导,从而导致故障信息利用率低,分析过程繁琐,所需故障样本量过大.

经过对以上攻击方法的分析,本研究将故障分析^[7-8]、代数分析^[9]和猜测确定攻击^[10]三者相结合,提出一种自动化、通用化、实用化的密码分析方法,并将其应用到 SOSEMANUK 流密码安全性分析中.在现有攻击的基础上,利用故障注入技术向密码注入随机单字故障,引入额外故障信息,并通过在密码多处注入故障,使故障信息扩展至密码所有初始内部状态,以弥补现有攻击因信息量不足而导致的攻击复杂度较高的不足,并在深入分析故障传播特征的基础上,利用代数分析方法将 SOSEMANUK 流密码和故障信息表示成代数方程组并使用 SAT 解析器进行自动化求解,恢复密码初始内部状态.实验结果表明:针对密码加密首轮攻击,20 次故障注入即可将攻击所需的计算复杂度减小到 $O(2^{96})$;针对密码前两轮加密攻击,10 次故障注入,无须猜测算法内部状态即可恢复全部初始状态,与现有攻击相比,本文的攻击方法有效降低了攻击复杂度.

1 SOSEMANUK 流密码攻击概述

SOSEMANUK 流密码由线性反馈移位寄存器(LFSR)、有限状态机(FSM)和 Serpent1 组成^[1].LFSR 长度为 10 级,在 $GF(2^{32})$ 上运算,内部状态自左向右进行移位,在 $t=0$ 时刻初始状态为 $(s_1, s_2, \dots, s_{10})$,以后每个时钟周期 LFSR 状态位按照公式

$$s_{t+10} = s_{t+9} \oplus \alpha^{-1} s_{t+3} \oplus \alpha s_t \quad (\forall t \geq 1) \quad (1)$$

进行更新,式中 α 和 s_t 的含义参见文献[1].

FSM 包含两个 32 bit 存储器,存储器的值分别用 R_1 和 R_2 表示,每个时钟周期以 $(s_{t+1}, s_{t+8}, s_{t+9})$ 作为输入,按照以下公式更新 R_1 和 R_2 ,并生成一个 32 bit 的输出 f_t ,即

$$R_{1t} = R_{2t-1} \boxplus (r_{t-1} s_{t+1} \oplus s_{t+8}); \quad (2)$$

$$R_{2t} = \text{trans}(R_{1t-1}); \quad (3)$$

$$f_t = (s_{t+9} \boxplus R_{1t}) \oplus R_{2t}; \quad (4)$$

$$\text{trans}(z) = (M \times z) \lll n, \quad (5)$$

式中:符号 $\oplus$ 表示异或运算;符号 $\boxplus$ 表示模 2^{32} 加运算; r_{t-1} 为 R_{1t-1} 的最低位;下标 $\lll n$ 表示将变量 $(M \times z)$ 循环左移 n 位; M 是一个值为 0x54655307 的常量.在 $t=0$ 时刻,密码算法指定内部状态初始值 $\Phi_0 = (s_1, s_2, \dots, s_{10}, R_{10}, R_{20})$,然后从 $t=1$ 开始每 4 个时钟周期生成 128 bit 的密钥流 $(z_{t+3}, z_{t+2}, z_{t+1}, z_t)$.为方便叙述,将密码算法每 4 个时钟周期生成 128 bit 密钥流这一过程称为一轮加密.

1.1 攻击基本过程

基于故障信息的 SOSEMANUK 流密码猜测确定攻击目标是恢复密码初始时刻的内部状态 Φ_0 ,攻击流程如下:

a. 随机选取初始密钥 K 和初始向量 I_{IV} 得到密码初始内部状态 Φ_0 ,使用 Φ_0 生成正确密钥流 Z ;

b. 在 $t=1$ 时刻,使用故障注入技术向密码注入随机单字故障,产生错误的密钥流 Z' ;

c. 对首轮加密进行攻击时猜测密码初始时刻内部状态 s_7, s_8 和 s_9 ,对前两轮加密进行攻击时则无须猜测内状态,使用代数分析方法构建密码等效代数方程组,并将故障信息用代数方程组表示,使用 SAT 解析器求解所构建的代数方程组,得到求解结果 Φ_0^* ;

d. 比较 Φ_0 和 Φ_0^* 是否一致,验证 Φ_0^* 的正确性.

1.2 故障模型设定

本研究采用面向单字的随机故障模型,其基本假设为:攻击者能够正确运行密码芯片生成正确的密文输出,并能够重启密码算法使其恢复至初始状态;同时,攻击者每次可在 $t=1$ 时刻,使用故障注入技术诱发密码 12 个内部状态中的某一个发生故障,生成错误的密文输出,其中具体的故障值未知,但故障发生的具体位置已知.

实际上,目前的故障注入技术按成本可分为高成本注入技术和低成本注入技术^[10],高成本技术通过光脉冲、激光束、聚焦离子束(FIB)等能够确保攻击者向密码芯片注入故障时,在时间和空间上具有相当高的精度.低成本技术则利用时钟毛刺、电压峰值等向密码芯片注入故障,无法在时间和空间上达到高成本技术的精度.文献[11-12]利用近红外激光故障注入系统成功向 DES 芯片

注入故障,并通过技术手段可以判定故障的具体位置.综上所述,本研究设定的故障模型是实际可行的.

1.3 故障传播特性分析

假设攻击者在 $t=1$ 时刻向密码 s_{t+4} 位置注入随机单字故障 $\gamma=(\xi_{31}, \xi_{30}, \dots, \xi_0)$,由故障传播可知^[11-12]:随着故障的传播,故障会导致 f_{t+3} 出错,且假设故障 γ 作用范围覆盖 f_{t+3} 所有比特,则故障最终可能会导致 S 盒输出比特全部出错.相应地,当故障注入位置为密码其他内部状态时,故障对 S 盒输入的影响情况如下(为方便论述,令 ϕ 表示故障注入位置, Δ_1 和 Δ_2 分别表示密码首轮加密和第二轮加密 S 盒可能输入差分, r_0 表示 R_{10} 最低位, $0x2, 0x4$ 等均表示十六进制数):

当 $\phi \in \{s_1, s_3, s_4\}$ 时, $\Delta_1 = \{0x2, 0x4, 0x6, 0x8, 0xa, 0xd, 0xe\}$, $\Delta_2 = \{0x1, 0x2, 0x3, 0x4, 0x5, 0x6, 0x7, 0x8, 0x9, 0xa, 0xb, 0xc, 0xd, 0xe, 0xf\}$;

当 $\phi \in \{s_2, s_{10}, R_{10}, R_{20}, s_9\}$ 且 $r_0 = 1$ 时, $\Delta_1 = \Delta_2 = \{0x1, 0x2, 0x3, 0x4, 0x5, 0x6, 0x7, 0x8, 0x9, 0xa, 0xb, 0xc, 0xd, 0xe, 0xf\}$;

当 $\phi \in \{s_5\}$ 时, $\Delta_1 = \{0x4, 0x8, 0xc\}$, $\Delta_2 = \{0x1, 0x2, 0x3, 0x4, 0x5, 0x6, 0x7, 0x8, 0x9, 0xa, 0xb, 0xc, 0xd, 0xe, 0xf\}$;

当 $\phi \in \{s_6\}$ 时, $\Delta_1 = \{0x8\}$, $\Delta_2 = \{0x1, 0x2, 0x3, 0x4, 0x5, 0x6, 0x7, 0x8, 0x9, 0xa, 0xb, 0xc, 0xd, 0xe, 0xf\}$;

当 $\phi \in \{s_7\}$ 时, $\Delta_1 = 0$, $\Delta_2 = \{0x1, 0x2, 0x3, 0x4, 0x5, 0x6, 0x7, 0x8, 0x9, 0xa, 0xb, 0xc, 0xd, 0xe, 0xf\}$;

当 $\phi \in \{s_8\}$ 时, $\Delta_1 = 0$, $\Delta_2 = \{0x4, 0x8, 0xc\}$;

当 $\phi \in \{s_9\}$ 且 $r_0 = 0$ 时, $\Delta_1 = 0$, $\Delta_2 = \{0x8\}$.

由上面分析可知,根据故障位置可以确定 S 盒输入差分的取值集合.

1.4 故障涉及的密码初始内部状态分析

记变量 κ 在密码运行过程中,通过关系式 $(*)$ 涉及的密码初始内部状态集合为 Σ ,简记为:

$$\kappa \xrightarrow{(*)} \Sigma.$$

记故障在前 k 轮加密传播过程中,涉及的所有密码初始内部状态的集合为 Ω ,即 $\Omega_k = \{s_m, R_{n0} | m, n \in \mathbb{Z}, \text{且 } 1 \leq m \leq 10, 1 \leq n \leq 2\}$,则易知 $(t \geq 1)$

$$s_{t+10} \xrightarrow{(1)} s_{t+9}, s_{t+3}, s_t; \quad (6)$$

$$R_{1t} \xrightarrow{(2)} R_{2t-1}, s_{t+1}, s_{t+8}; \quad (7)$$

$$R_{1t} \xrightarrow{(2)} R_{2t-1}, s_{t+1}; \quad (8)$$

$$R_{2t} \xrightarrow{(3)} R_{1t-1}; \quad (9)$$

$$f_t \xrightarrow{(4)} R_{1t}, R_{2t}, s_{t+9}. \quad (10)$$

由式(6)~(10)可得

$$\left\{ \begin{array}{l} f_1, f_2, f_3, f_4 \xrightarrow{(4)(6)(7)(8)} \\ \{R_{10}, R_{20}, s_1, s_2, s_3, s_4, s_5, s_6, s_9, s_{10}\} \\ (r_0 = 1); \\ \{R_{10}, R_{20}, s_1, s_2, s_3, s_4, s_5, s_6, s_{10}\} \\ (r_0 = 0). \end{array} \right. \quad (11)$$

同理可得

$$f_1, f_2, f_3, f_4, f_5, f_6, f_7 \xrightarrow{(4)(6)(7)(8)} \Phi_0. \quad (12)$$

由上述分析可知:当攻击者对密码首轮加密进行攻击,注入的故障影响为 f_1, f_2, f_3, f_4 时,故障信息涉及除 s_7, s_8 和 s_9 外的密码所有初始内部状态,理论上只须猜测 s_7, s_8 和 s_9 三个内部状态即可恢复 Φ_0 ;对前两轮加密进行攻击时,当注入的故障影响 $f_1, f_2, f_3, f_4, f_5, f_6, f_7$ 时,理论上无须猜测内部状态即可恢复 Φ_0 .

2 SOSEMANUK 流密码代数方程组构建

SOSEMANUK 流密码等效代数方程组构建过程中,关键是构造 LFSR 状态更新函数、S 盒、模 2^{32} 加和 $\text{trans}(\cdot)$ 等非线性组件在比特层面的等效代数方程组,下面具体给出构建方法.

2.1 LFSR 状态更新函数代数方程组构建

由于变量 $\alpha s_t, \alpha^{-1} s_{t+3}, s_{t+10}, s_{t+9} \in \text{GF}(2^{32})$,因此令 $s_{t+10} = (\omega_{31}, \omega_{30}, \dots, \omega_0)$, $s_{t+9} = (\tau_{31}, \tau_{30}, \dots, \tau_0)$, $\alpha s_t = (\mu_{31}, \mu_{30}, \dots, \mu_0)$, $\alpha^{-1} s_{t+3} = (\lambda_{31}, \lambda_{30}, \dots, \lambda_0)$, $y_0, x_0, \mu_0, \lambda_0$ 分别表示最低位,于是可将式(1)转化成比特层面的数学关系式(s_{t+10} 每个比特的关系式)如下所示.

$$\begin{aligned} \omega_0 &= \tau_0 + \mu_{24} + \mu_{28} + \mu_{29} + \mu_{30} + \mu_{31} + \lambda_0 + \lambda_1 + \\ &\lambda_4 + \lambda_7 + \lambda_8; \omega_1 = \tau_1 + \mu_{24} + \mu_{25} + \mu_{29} + \mu_{30} + \mu_{31} + \\ &\lambda_1 + \lambda_2 + \lambda_5 + \lambda_9; \omega_2 = \tau_2 + \mu_{25} + \mu_{26} + \mu_{30} + \mu_{31} + \\ &\lambda_0 + \lambda_2 + \lambda_3 + \lambda_6 + \lambda_{10}; \omega_3 = \tau_3 + \mu_{26} + \mu_{27} + \mu_{28} + \\ &\mu_{29} + \mu_{30} + \lambda_0 + \lambda_3 + \lambda_{11}; \omega_4 = \tau_4 + \mu_{24} + \mu_{27} + \mu_{28} + \\ &\mu_{29} + \mu_{30} + \mu_{31} + \lambda_1 + \lambda_4 + \lambda_{12}; \omega_5 = \tau_5 + \mu_{25} + \lambda_1 + \\ &\lambda_2 + \lambda_4 + \lambda_5 + \lambda_7 + \lambda_{13}; \omega_6 = \tau_6 + \mu_{26} + \lambda_0 + \lambda_2 + \lambda_3 + \\ &\lambda_5 + \lambda_6 + \lambda_{14}; \omega_7 = \tau_7 + \mu_{27} + \mu_{28} + \mu_{29} + \mu_{30} + \mu_{31} + \\ &\lambda_0 + \lambda_3 + \lambda_6 + \lambda_{15}; \omega_8 = \tau_8 + \mu_0 + \mu_{24} + \mu_{25} + \mu_{28} + \\ &\lambda_2 + \lambda_3 + \lambda_4 + \lambda_6 + \lambda_7 + \lambda_{16}; \omega_9 = \tau_9 + \mu_1 + \mu_{24} + \\ &\mu_{25} + \mu_{26} + \mu_{29} + \lambda_3 + \lambda_4 + \lambda_5 + \lambda_7 + \lambda_{17}; \omega_{10} = \tau_{10} + \\ &\mu_2 + \mu_{24} + \mu_{25} + \mu_{26} + \mu_{27} + \mu_{30} + \lambda_4 + \lambda_5 + \lambda_6 + \lambda_{18}; \end{aligned}$$

$\omega_{11} = \tau_{11} + \mu_3 + \mu_{24} + \mu_{26} + \mu_{27} + \mu_{31} + \lambda_2 + \lambda_3 + \lambda_4 + \lambda_5 + \lambda_{19}; \omega_{12} = \tau_{12} + \mu_4 + \mu_{25} + \mu_{27} + \mu_{28} + \lambda_3 + \lambda_4 + \lambda_5 + \lambda_6 + \lambda_{20}; \omega_{13} = \tau_{13} + \mu_5 + \mu_{25} + \mu_{26} + \mu_{29} + \lambda_2 + \lambda_3 + \lambda_5 + \lambda_{21}; \omega_{14} = \tau_{14} + \mu_6 + \mu_{24} + \mu_{26} + \mu_{27} + \mu_{30} + \lambda_0 + \lambda_3 + \lambda_4 + \lambda_6 + \lambda_{22}; \omega_{15} = \tau_{15} + \mu_7 + \mu_{24} + \mu_{27} + \mu_{31} + \lambda_1 + \lambda_2 + \lambda_3 + \lambda_5 + \lambda_6 + \lambda_{23}; \omega_{16} = \tau_{16} + \mu_8 + \mu_{24} + \mu_{25} + \mu_{26} + \mu_{27} + \mu_{28} + \mu_{29} + \mu_{31} + \lambda_0 + \lambda_5 + \lambda_7 + \lambda_{24}; \omega_{17} = \tau_{17} + \mu_9 + \mu_{24} + \mu_{25} + \mu_{26} + \mu_{27} + \mu_{28} + \mu_{29} + \mu_{30} + \lambda_0 + \lambda_1 + \lambda_6 + \lambda_{25}; \omega_{18} = \tau_{18} + \mu_{10} + \mu_{24} + \mu_{25} + \mu_{26} + \mu_{27} + \mu_{28} + \mu_{29} + \mu_{30} + \mu_{31} + \lambda_0 + \lambda_1 + \lambda_2 + \lambda_7 + \lambda_{26}; \omega_{19} = \tau_{19} + \mu_{11} + \mu_{24} + \mu_{30} + \lambda_0 + \lambda_1 + \lambda_2 + \lambda_3 + \lambda_5 + \lambda_7 + \lambda_{27}; \omega_{20} = \tau_{20} + \mu_{12} + \mu_{24} + \mu_{25} + \mu_{31} + \lambda_1 + \lambda_2 + \lambda_3 + \lambda_4 + \lambda_6 + \lambda_{28}; \omega_{21} = \tau_{21} + \mu_{13} + \mu_{27} + \mu_{28} + \mu_{29} + \mu_{31} + \lambda_2 + \lambda_3 + \lambda_4 + \lambda_{29}; \omega_{22} = \tau_{22} + \mu_{14} + \mu_{28} + \mu_{29} + \mu_{30} + \lambda_3 + \lambda_4 + \lambda_5 + \lambda_{30}; \omega_{23} = \tau_{23} + \mu_{15} + \mu_{24} + \mu_{25} + \mu_{26} + \mu_{27} + \mu_{28} + \mu_{30} + \lambda_4 + \lambda_6 + \lambda_7 + \lambda_{31}; \omega_{24} = \tau_{24} + \mu_{16} + \mu_{24} + \mu_{25} + \mu_{27} + \lambda_4 + \lambda_7; \omega_{25} = \tau_{25} + \mu_{17} + \mu_{25} + \mu_{26} + \mu_{28} + \lambda_5; \omega_{26} = \tau_{26} + \mu_{18} + \mu_{26} + \mu_{27} + \mu_{29} + \lambda_6; \omega_{27} = \tau_{27} + \mu_{19} + \mu_{25} + \mu_{28} + \mu_{30} + \lambda_0 + \lambda_4; \omega_{28} = \tau_{28} + \mu_{20} + \mu_{26} + \mu_{29} + \mu_{31} + \lambda_0 + \lambda_1 + \lambda_5; \omega_{29} = \tau_{29} + \mu_{21} + \mu_{24} + \mu_{25} + \mu_{30} + \lambda_1 + \lambda_2 + \lambda_4 + \lambda_6 + \lambda_7; \omega_{30} = \tau_{30} + \mu_{22} + \mu_{24} + \mu_{25} + \mu_{26} + \mu_{31} + \lambda_2 + \lambda_3 + \lambda_5 + \lambda_7; \omega_{31} = \tau_{31} + \mu_{23} + \mu_{24} + \mu_{26} + \lambda_3 + \lambda_6 + \lambda_7.$

2.2 S 盒代数方程组构建

定义 S 盒的输入为 X 和 Y , 输出为 Z , $X = (x_3, x_2, x_1, x_0)$, $Y = (y_3, y_2, y_1, y_0)$, $Z = (z_3, z_2, z_1, z_0)$, x_0, y_0, z_0 分别为 X, Y, Z 的最低位, 则基于布尔积理论 S 盒可以表示为

$$\begin{cases}
 y_0 = x_1 \oplus x_2 \oplus x_3 \oplus x_0 x_2; \\
 y_1 = x_0 \oplus x_1 \oplus x_2 \oplus x_0 x_3 \oplus x_1 x_2 \oplus x_2 x_3 \oplus x_0 x_1 x_2 \oplus x_0 x_1 x_3 \oplus x_0 x_2 x_3; \\
 y_2 = x_0 \oplus x_1 \oplus x_3 \oplus x_1 x_2 \oplus x_1 x_3 \oplus x_2 x_3 \oplus x_0 x_1 x_3 \oplus x_0 x_2 x_3; \\
 y_3 = x_0 \oplus x_1 \oplus x_2 \oplus x_1 x_3 \oplus x_0 x_1 x_2 \oplus 1.
 \end{cases} \quad (13)$$

2.3 模 2^{32} 加运算代数方程组构建

定义 $X, Y, Z \in \text{GF}(2^n)$, $X = (x_{n-1}, x_{n-2}, \dots, x_0)$, $Y = (y_{n-1}, y_{n-2}, \dots, y_0)$, $Z = (z_{n-1}, z_{n-2}, \dots, z_0)$, 则 $\text{GF}(2^n)$ 上的模 2^n 加运算可表示为:

$$\begin{aligned}
 z_0 &= x_0 \oplus y_0; \\
 z_1 &= x_1 \oplus y_1 \oplus x_0 y_0; \\
 z_2 &= x_2 \oplus y_2 \oplus x_1 y_1 \oplus (x_1 \oplus y_1)(x_1 \oplus y_1 \oplus z_1); \\
 &\dots \\
 z_i &= x_i \oplus y_i \oplus x_{i-1} y_{i-1} \oplus (x_{i-1} \oplus
 \end{aligned}$$

$$y_{i-1})(x_{i-1} \oplus y_{i-1} \oplus z_{i-1});$$

...

$$\begin{aligned}
 z_{n-1} &= x_{n-1} \oplus y_{n-1} \oplus x_{n-2} y_{n-2} \oplus \\
 &(x_{n-2} \oplus y_{n-2})(x_{n-2} \oplus y_{n-2} \oplus z_{n-2}).
 \end{aligned}$$

2.4 trans(·) 运算代数方程组构建

式(5)中的 $\text{trans}(\cdot)$ 运算可等效转化为,

$$\text{trans}(z) = (M \times z)_{\ll 7} =$$

$$(0x54655307 \times z)_{\ll 7} =$$

$$((01010100011001010101001100000111)_2 \times$$

$$z)_{\ll 7} = \bigoplus_{i \in \Gamma} (z)_{\ll i} \ll 7, \quad (14)$$

式中 $\Gamma = \{30, 28, 26, 22, 21, 18, 16, 14, 12, 9, 8, 2, 1, 0\}$.

2.5 故障信息表示

由于对 SOSEMANUK 流密码首轮和前两轮加密攻击故障信息表示方法相同, 因此下面以对密码首轮加密进行攻击, 以故障位置为 s_5 时的情形为例, 给出故障信息表示方法, 其他故障位置的故障信息表示与此类似.

A. 故障位置信息表示

设注入故障前 $s_5 = (x_{31}, x_{30}, \dots, x_0)$, 注入故障后 $s'_5 = (y_{31}, y_{30}, \dots, y_0)$, 则故障 $\gamma = (\xi_{31}, \xi_{30}, \dots, \xi_0)$ 可表示为

$$\xi_i = x_i \oplus y_i \quad (0 \leq i \leq 31), \quad (15)$$

由于只在 s_5 注入故障, 因此有:

$$\begin{aligned}
 s_i &= s'_i \quad (i \in [1, 4] \cup [6, 10]); \\
 R_{n0} &= R'_{n0} \quad (n \in [0, 1]).
 \end{aligned} \quad (16)$$

将式(15)和(16)直接带入正确密文和故障密文样本.

B. 故障差分信息表示

由 1.3 节分析知, 当故障位置为 s_5 时, S 盒输入差分集合 $\Delta_1 = \{0x4, 0x8, 0xc\}$, 定义 $\Delta_{\text{out}} = \bigcup_{\Delta \in \Delta_1} S(X) \oplus S(X \oplus \Delta)$ 为 S 盒输出差分集合, 则根据 S 盒分析可得 $\Delta_{\text{out}} = \{0x2, 0x3, 0x4, 0x5, 0x6, 0x7, 0x8, 0x9, 0xa, 0xb, 0xd, 0xe, 0xf\}$.

设 $\beta_n (0 \leq n \leq N(\Delta_{\text{out}}) - 1)$ 为集合 Δ_{out} 中的元素, 其中 $N(\Delta_{\text{out}})$ 表示 Δ_{out} 集合中的元素个数, 则 $\forall \beta_n \in \Delta_{\text{out}}$, 定义集合

$$\begin{aligned}
 \Gamma_n &= \{X'_i \mid S(X_i) \oplus S(X'_i) = \beta_n\} \\
 &(0 \leq i \leq 31),
 \end{aligned}$$

在 $t=1$ 时刻, 第 i 个 S 盒的正确输入可表示为

$$\begin{aligned}
 f_k^i &= \prod_{m=0}^{N(\Gamma_n)-1} (f_k^i)'_m [(f_k^i)'_{N(\Gamma_n)-1} = \\
 &(f_k^i)'_{N(\Gamma_n)-2} \dots = (f_k^i)'_0, 1 \leq k \leq 4]. \quad (17)
 \end{aligned}$$

例如, $\beta_2 = 0x3$, 相应的 $\Gamma_2 = \{0x4, 0xc\}$, 则由式(17)可知: $f_3^i = 1, f_2^i = 0, f_1^i = 0$.

3 攻击复杂度分析

基于故障信息的 SOSEMANUK 流密码猜测确定攻击复杂度主要由故障注入总数和计算复杂度二者构成。

3.1 故障注入个数

由 1.4 节分析可知,当攻击者对密码首轮加密进行攻击,注入的随机故障覆盖 f_1, f_2, f_3, f_4 时,理论上攻击者仅须猜测 s_7, s_8 和 s_9 并求解代数方程组即可恢复 Φ_0 . 此时,将故障覆盖的 S 盒输入比特个数为 k 的概率记为 $P[k]$,则有:

当 $\phi \in \{s_7, s_8, s_9 (r_0 = 0)\}$ 时,故障不影响任何 S 盒输入, $P(k=0) = 5/24$;

当 $\phi \in \{R_{10}, R_{20}, s_2, s_{10}, s_9 (r_0 = 1)\}$ 时,故障覆盖 f_1, f_2, f_3, f_4 , $P(k=4) = 9/24$;

当 $\phi \in \{s_1, s_3, s_4\}$ 时,故障覆盖 f_2, f_3, f_4 , $P(k=3) = 3/12$;

当 $\phi \in \{s_5\}$ 时,故障覆盖 f_3, f_4 , $P(k=2) = 1/12$;

当 $\phi \in \{s_6\}$ 时,故障覆盖 f_4 , $P(k=1) = 1/12$.

由上述分析可知:注入 n 次故障,故障覆盖 f_1, f_2, f_3, f_4 的概率 $P = \sum_{i=1}^n (1 - P(k=4))^{i-1} \times (P(k=4))$. 理论上来说,使得 $P \rightarrow 1$ 对应的 n 为理论所需的故障注入个数,即 $n=20$.

同理,当攻击者对 SOSEMANUK 流密码前两轮加密进行攻击时,故障注入个数 $n=10$.

3.2 计算复杂度

当攻击者对 SOSEMANUK 流密码首加密进行攻击时,由于攻击者并不知道密码内部状态 R_{10} 最低位 r_0 的值,所以须要猜测三个 32 bit 初始内部状态变量 s_7, s_8 和 s_9 ,故其时间复杂度为 $O(2^{96})$. 对密码前两轮加密进行攻击时,则无须猜测密码内部状态即可恢复 Φ_0 .

4 攻击实验及分析比较

本研究主要对故障注入之后的离线分析方法进行研究,在线故障注入不是本研究的研究重点,故在实验仿真中使用 C++ 编程语言软件模拟实现随机单字故障注入(CPU 为 Intel(R), Core(TM) i7-4790 3.60 GHz,内存 12 G, Windows 7 64 bit 操作系统),使用 CryptoMinisat 4.4 解析器进行求解,该解析器支持输出多解(解析器详细使用方法见文献[12]). 为规范攻击成功率,当解

析器求解方程时间大于 3 600 s 或输出解不唯一时,视为攻击失败.

4.1 SOSEMANUK 流密码首轮加密攻击实例

下面给出一个 SOSEMANUK 流密码代数故障攻击实例.

a. 使用初始密钥 $K = 0x00112233445566778899AABBCCDDEEFF$, 初始向量 $I_{IV} = 0x8899AABBCCDDEEFF0011223344556677$, 生成初始内部状态 $\Phi_0 = (0x101D095B, 0x9AB17002, 0xA93D885C, 0xABB5043B, 0xC0AE3CD2, 0x6076C06F, 0xDBA99A9F, 0xCEC69B9B, 0x1BFABB41, 0x21A53B2B, 0x4F9F2596, 0xB3A3B696)$, 使用 2.1 节方法构建正确加密时密码完整等效代数方程组.

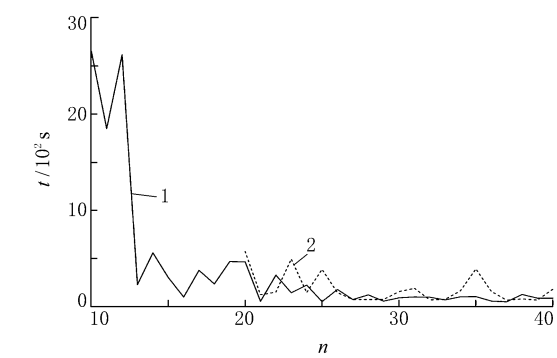
b. 猜测 s_7, s_8 和 s_9 的值,并在 $t=1$ 时刻向密码任意位置注入宽度为一个字的随机故障,在分析故障传播特征的基础上,利用 3.2 节方法将故障信息用代数方程组表示出来.

c. 将构建的代数方程组转化 SAT 字句,使用 CryptoMinisat 4.4 解析器对上述构建的代数方程组进行求解得到唯一解 Φ'_0 , 转化为 16 进制与 Φ_0 相同,攻击成功,需要 34 次故障注入,耗时 78.08 s.

4.2 实验结果分析与比较

实验中,对密码首轮和前两轮加密的攻击实验均执行了 100 次,结果如图 1 所示(t 为求解时间). 由图 1 可知:对密码加密首轮攻击所需故障注入次数最少为 20 次,平均求解时间为 271.98 s. 当故障个数小于 20 次时,使用解析器求解所构建的代数方程组解的个数不唯一,视为攻击失败. 对密码前两轮加密攻击所需最少的故障注入次数为 10 次,平均求解时间为 2 652.40 s. 当故障注入次数少于 10 次时,方程的平均求解时间大于 3 600 s,视为攻击失败. 对上述实验结果分析可知,对密码首轮和前两轮加密攻击所需故障注入次数分别为 20 次和 10 次,实验结果与 2.3 节理论分析结果一致,验证了攻击方法的正确性.

对密码首轮进行攻击时,须要猜测三个 32 bit 内部变量. 对于本研究,当猜测错误时,构造的代数方程组出现错误,方程组不可解, SAT 解析器会在较短时间 t_1 内输出“UNSATISFABLE”^[12],从而导致攻击失败,当猜测正确,构造的代数方程组正确无误时, SAT 解析器则会在 t_2 时间内输出满足方程的解,且 $t_2 \gg t_1$. 基于此,本研究使用 SAT 解析器作为区分器,对错误的猜测进行排除. 表 1 为对密码加密首轮进行攻击时,不



1—对前两轮进行攻击; 2—对首轮进行攻击.

图 1 对密码攻击实验结果

同故障注入次数下猜测错误时解析器的平均求解时间(部分结果). 由表 1 可知:与猜测正确时解析器的平均求解时间相比(图 1),猜测错误时解析器的平均求解时间非常小.

表 1 攻击实验结果比较		
编号	故障个数	t/s
1	20	0.88
2	21	0.9
3	22	0.96
...	...	...
18	38	1.37
19	39	1.35
20	40	1.45

4.3 实验结果比较

与文献[3-5]相比,本研究充分利用了密码差分 S 盒分布特性,使攻击所需的计算复杂度显著降低,针对密码加密首轮攻击,攻击所需的计算复杂度为 $O(2^{96})$,针对密码前两轮加密攻击,则无须猜测算法内部状态即可恢复全部初始状态. 与文献[6]相比,本研究在多个故障点注入故障,使故障信息涉及全部 Φ_0 ,理论上能够完全恢复 Φ_0 ,针对密码加密首轮攻击所需的故障注入个数为 20,针对密码前两轮加密攻击所需猜故障个数为 10. 此外,本研究将密码算法及故障信息采用代数方程的形式进行表示,并使用 SAT 解析器进行自动化求解,有效弥补了文献[6]分析过程复杂,故障信息利用率低,所需样本量大的不足,增强了攻击方法的有效性.

实验结果表明:基于故障信息的猜测确定攻击能够显著降低现有攻击方法的复杂度. 此外,本

研究可为其他具有类似结构的密码算法的安全性分析提供一定的参考.

参 考 文 献

[1] Berbain C, Billet O, Canteaut A, et al. SOSEMANUK: a fast software-oriented stream cipher [C]// New Stream Cipher Designs, LNCS 4986. Berlin: Springer, 2008: 98-118.

[2] Tsunoo Y, Saito T, Shigeri M, et al. Evaluation of SOSEMANUK with regard to guess and determine attacks[EB/OL]. [2016-01-02]. <http://www.ecrypt.eu.org/stream/paperdir/2006.009.pdf>.

[3] Feng X, Liu J, Zhou Z, Wu C K, et al. A byte-based guess and determine attack on SOSEMANUK[C]// ASIACRYPT 2010, LNCS 6477. Berlin: Springer, 2010:146-157.

[4] 谢端强,李恒,李瑞林,等. 对 Sosemanuk 算法改进的猜测决定攻击[J]. 国防科技大学学报, 2012, 34(6): 79-83.

[5] Esmaeili S Y, Kircanski A, Youssef A, Differential fault analysis of sosemanuk[C]// AFRICACRYPT 2011, LNCS 6737. Berlin: Springer, 2011: 316-331

[6] Boneh D, DeMillo R A, Lipton R J. On the Importance of checking cryptographic protocols for faults [C] // EUROCRYPT 1997, LNCS 1233. Berlin: Springer, 1997: 37-51.

[7] Biehl I, Mwyer B, Muller V. Differential fault analysis on elliptic curve cryptosystems[C] // CRYPTO 2000, LNCS 1880. Berlin: Springer, 2000: 131-146.

[8] Courtois N, Pieprzyk J. Cryptanalysis of block ciphers with overdefined systems of equations[C]// ASIACRYPT 2002, LNCS 2501. Berlin: Springer, 2002: 267-287.

[9] 关杰,丁林,刘树凯. SNOW3G 与 ZUC 流密码的猜测确定攻击[J]. 软件学报, 2013, 24(6): 1324-1333.

[10] Marc J, Michale T. Fault analysis in cryptography [M]. Berlin: Spring-Verlag Heidelberg, 2012.

[11] 刘辉志,赵东艳,张海峰,等. 近红外激光故障注入系统在密码芯片攻击中的应用[J]. 科学技术与工程, 2014, 14(22): 225-230.

[12] Soos M, Nohl K, Castelluccia C. Extending SAT solvers to cryptographic problems[C]// SAT 2009, LNCS 5584. Berlin: Springer, 2009:244-257.